Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- İkili ağaçta dolaşma (tree traversal) yöntemlerinin tekrarı: preorder, inorder ve postorder. Postorder dolaşmanın özyinelemeli (recursive) kodu.
- Sıralı bir bağlı listede (linked list) aramanın neden $O(n)$ olduğu ve ikili arama ağacının (binary search tree, BST) bu sorunu nasıl çözdüğü.
- İkili ağacın yüksekliği (height) ile yaprak ve düğüm sayıları arasındaki ilişkiler. Dengeli (balanced) ve dengesiz ağaç.
- İkili arama ağacında ekleme (insertion), silme (deletion) ve arama (search). Düğüm yapısı, `newNode`, `insert` ve `search` fonksiyonları.
- İkili ağacın dizide saklanması ve seyrek ağaçlarda boş göz sorunu.
- Bellekteki heap bölgesi ile heap ağacı (heap tree) arasındaki fark.
- Heap ağacının üç özelliği: dengeli olma, sola dayalı olma (left justified) ve heap özelliği (max heap, min heap).
- Heap ağacına eleman ekleme (sift up), kökü silme (sift down) ve heapify işlemi.
- Heap sort algoritması, adım adım örnekler ve karmaşıklığı.
- Önceki quizin bazı sorularının çözümü: dairesel kuyrukta yeni elemanın yeri ve yığın (stack) ile kuyruk (queue) gerçekleştirme.

Hocanın Özellikle Vurguladığı Kısımlar

- Özyinelemeli bir algoritma yazarken ilk bilmeniz gereken şey durma noktasıdır. Ağaçta durma noktası, bakılan düğümün `NULL` olmasıdır.
- İkili arama ağacında esas amaç aramadır. Ağaç dengeli ise arama $O(\log n)$, tek tarafa yığılmış (dengesiz) ise $O(n)$ olur.
- Ağaç zamanla dengesini kaybedebilir. Belli aralıklarla ağacı yeniden dengelemek doğru bir yaklaşımdır.
- Yeni düğüm ikili arama ağacında her zaman bir yaprağa eklenir. Araya eleman sokmaya çalışmak zahmetlidir.
- `malloc` ile yer ayrıldıktan sonra yerin gerçekten alınıp alınmadığı her zaman kontrol edilmelidir.
- Dizi, aralarda çok boşluk olmayacağından emin olduğunuz durumlarda bağlı listeden daha verimlidir. Çünkü istediğiniz göze tek adımda erişirsiniz.
- Özyinelemeyi gereksiz yere kullanmayın, fonksiyon çağırmak sisteme yük getirir. Ama ağaç işlemlerinde özyinelemeli çözüm çok daha kolaydır.
- Heap ağacı dengeli ve sola dayalı olduğu için arada boş göz kalmaz. Bu yüzden bir dizide rahatça saklanabilir.
- Heap ağacında işlemler genellikle kök üzerinden yapılır. Kökte ya en küçük ya en büyük eleman vardır.
- Sift down sırasında aşağı inen eleman, iki çocuğun büyüğü (max heap) ya da küçüğü (min heap) ile karşılaştırılır. Tek bir çocuğa bakmak yanlış sonuç verir.
- Hoca, ikili arama ağacında silme fonksiyonunu kendi başınıza yazmayı pratik olarak önerdi.

Kısa Tekrar Notları

- **Preorder:** kök, sol alt ağaç, sağ alt ağaç. `+ a b` (prefix).

- **Inorder:** sol alt ağaç, kök, sağ alt ağaç. `a + b` (infix).
- **Postorder:** sol alt ağaç, sağ alt ağaç, kök. `a b +` (postfix).
- İkili arama ağacını inorder dolaşırsanız elemanları küçükten büyüğe sıralı görürsünüz.
- **BST özelliği:** kökteki değer x ise sol alt ağaçtaki bütün değerler x'ten küçük, sağ alt ağaçtaki bütün değerler x'ten büyük veya eşittir.
- Yüksekliği h olan ikili ağaç (kök 0. seviye):
  - En fazla yaprak sayısı: 2^h.
  - En fazla düğüm sayısı: $2^{h+1} - 1$.
  - En az yaprak sayısı: 1.
  - En az düğüm sayısı: $h + 1$.
- n düğümlü ağaç dengeli ise yüksekliği $O(\log_2 n)$'dir.
- **BST'den silme, üç durum:**
  - Yaprak: ebeveynin işaretçisi `NULL` yapılır.
  - Tek çocuk: ebeveyn, silinen düğüm yerine onun çocuğunu gösterir.
  - İki çocuk: sağ alt ağacın en küçük elemanı (ya da sol alt ağacın en büyüğü) silinen düğümün yerine getirilir.
- **Heap ağacı:** dengeli + sola dayalı + heap özelliği.
- **Max heap:** her ebeveyn çocuklarından büyüktür. **Min heap:** her ebeveyn çocuklarından küçüktür. Sol ve sağ çocuk arasında bir sıra kuralı yoktur.
- Dizide i. gözdeki düğümün çocukları $2i + 1$ ve $2i + 2$. gözlerdedir.
- Sift up ve sift down: $O(\log n)$. Ekleme yoluyla sıfırdan heap kurma: $O(n \log n)$. Heap sort: $O(n \log n)$.

Detaylı Açıklamalar

Dolaşma Yöntemlerinin Tekrarı

- Ağaç hiyerarşik yapılar için uygundur: karar ağaçları, işletim sistemindeki dizin (directory) yapısı gibi.
- Her alt ağaç (subtree) kendi içinde bir ağaçtır. Kökün solu bitmeden sağa geçilmez. Bu yüzden aynı iş her alt ağaçta tekrar eder ve çözüm doğal olarak özyinelemelidir.
- Örnek ifade $a * b + c$ için ağaç: kökte `+`, solda `*` (çocukları `a` ve `b`), sağda `c`. Postorder dolaşma `a b * c +` verir.
- Postorder kodu (ekrandan, 00:13:30):

```
void postOrder(NODE *treeNode)
{
    if (treeNode != NULL)
    {
        postOrder(treeNode->left);
        postOrder(treeNode->right);
        printf("%d\n", treeNode->value);
    }
}
```

- Preorder'da `printf` iki çağrıdan önce gelir. Inorder kodunu yazmak öğrencilere bırakıldı.

Neden İkili Arama Ağacı?

- Sıralı bir dizide ikili arama (binary search) ile $O(\log n)$ arama yapılır. Çünkü ortadaki göze tek adımda erişilir.
- Sıralı bir bağlı listede ortadaki elemana erişmek için baştan itibaren gitmek gerekir. Arama yine $O(n)$ olur. Sıralı olmanın bir faydası kalmaz.
- Eleman sayısı sürekli değişiyorsa (3 eleman, sonra 30, sonra 7) statik dizi istemezsiniz. Bu durumda elemanları bağlı yapıda ama ikili arama ağacı mantığıyla saklarsınız.
- Böylece hem bağlı yapının esnekliği hem de $O(\log n)$ arama elde edilir. Her karşılaştırmada ağacın yarısı elenir (böl ve yönet, divide and conquer).
- Örnek ağaç (ekrandan, 00:22:50): kök 17. Sol alt ağaç 11 (çocukları 8 ve 14; 8'in çocukları 4 ve 9; 14'ün çocukları 12 ve 16). Sağ alt ağaç 21 (çocukları 20 ve 25; 25'in sağ çocuğu 33).

- $x = 14$ aranırken: $14 < 17$, sola git ve sağ tarafı tamamen ele. $14 > 11$, sağa git. 14 bulundu.

Yükseklik ve Denge

- Örnekte 0, 1 ve 2. seviyeleri tam dolu ağacın yüksekliği 2'dir. En fazla 4 yaprak ve 7 düğüm olabilir.
- Her seviyede tek düğüm olan ağaç (örneğin 15, 9, 7, 2 sırayla hep sola eklenmiş) dengesizdir. 4 düğüm vardır ama yükseklik 3'tür.
- Böyle bir ağaç pratikte bir bağlı listeye dönüşür. Arama ve ekleme en kötü durumda $O(n)$ olur.

Ekleme (Insertion)

- Önce yeni değerin yeri bulunur: kökten başlanır, küçükse sola, büyük veya eşitse sağa gidilir.
- Yer bulununca yeni düğüm yaprak olarak eklenir.
- Örnek (ekrandan, 00:37:25): kök 8; 8'in çocukları 5 ve 15; 5'in çocukları 3 ve 7; 15'in çocukları 10 ve 17.
  - 6 ekle: $6 < 8$ sola, $6 > 5$ sağa, $6 < 7$ sola. 6, 7'nin sol çocuğu olur.
  - 12 ekle: $12 > 8$ sağa, $12 < 15$ sola, $12 > 10$ sağa. 12, 10'un sağ çocuğu olur.
- Dengeli ağaçta ekleme $O(\log n)$'dir. Tek tarafa yığılmış ağaçta en sona eklemek $O(n)$'dir.

Silme (Deletion)

Örnekler ekrandan alınmıştır (00:47:30).

- **Durum 1, yaprak silme:** Ağaç 50 (çocukları 30 ve 70), 70'in sol çocuğu 60. 60 silinir. 70'in sol işaretçisi artık `NULL` gösterir.
- **Durum 2, tek çocuklu düğüm:** Ağaç 50 (çocukları 30 ve 70), 30'un tek çocuğu 40, 70'in çocukları 60 ve 80. 30 silinir. 50'nin sol işaretçisi artık 40'ı gösterir. 30'un yeri `free` ile serbest bırakılır.
- **Durum 3, iki çocuklu düğüm:** Ağaç 50 (çocukları 40 ve 70), 70'in çocukları 60 ve 80. 50 silinir.
  - Sağ alt ağacın en küçük elemanı bulunur: 60.
  - 60 köke alınır. Yeni ağaç: kök 60, solda 40, sağda 70 (70'in sağ çocuğu 80).
  - Kök değiştiği için ağacın başını tutan işaretçi de artık 60'ı gösterir.
- Aynı ağaçta 70 silinirse sağ alt ağaçta yalnız 80 olduğu için 80, 70'in yerine geçer. 70'in sol çocuğu 60 da 80'in sol çocuğu olur.
- Kökün yerine ya sol alt ağacın en büyüğü ya da sağ alt ağacın en küçüğü gelmelidir. Ancak böylece BST özelliği korunur. Genelde sağın en küçüğü seçilir.

Düğüm Yapısı, Ekleme ve Arama Kodları

Kod tahtadan alınmıştır (ekrandan, 01:14:30 ve 01:32:00).

```c
#include <stdlib.h>

typedef struct node
{
    int value;
    struct node *left;
    struct node *right;
} NODE;

NODE *newNode(int x)
{
    NODE *tmp;
    tmp = (NODE *)malloc(sizeof(NODE));
    tmp->value = x;
    tmp->left = NULL;
```

```
    tmp->right = NULL;
    return tmp;
}

NODE *insert(NODE *root, int x)
{
    if (root == NULL)            /* ağaç boş ise */
        return newNode(x);
    if (x < root->value)
        root->left = insert(root->left, x);
    else
        root->right = insert(root->right, x);
    return root;
}

NODE *search(NODE *root, int x)
{
    if (root == NULL || root->value == x)
        return root;
    else if (x < root->value)
        return search(root->left, x);
    else
        return search(root->right, x);
}
```

- `value` yerine birden çok bilgi içeren bir `struct` da olabilir. Kolaylık için `int` kullanıldı.
- Ekleme örneği: ağaç 15 (sol 11, sağ 17), 9 ekleniyor.
  - `insert(root, 9)`: $9 < 15$, `insert(root->left, 9)` çağrılır.
  - 11'de: $9 < 11$, `insert(NULL, 9)` çağrılır.
  - `root` artık `NULL` olduğundan `newNode(9)` yer ayırır (örnekte adres 1100) ve bu adres döner.
  - Dönen adres 11'in `left` alanına atanır. Böylece 9 ağaca bağlanır.
- Ana programda başta `root` `NULL`'dır. İlk eklemede dönen adres `root`'a atanır. Sonraki elemanlar aynı `root` ile `insert` çağrılarak eklenir.
- `search` iki durumda hemen `root`'u döndürür: ağaç (alt ağaç) boşsa ya da aranan değer köktedir. Çağıran taraf `NULL` gelirse "yok" der.
- Bu üç fonksiyon küçük bir `main` ile ajan tarafından `gcc -Wall` ile derlenip test edildi: 15, 17, 11, 9 eklendi; postorder çıktı `9 11 17 15`; 9 bulundu, 16 bulunamadı. Hoca derste bu eklemeleri bir program parçası olarak anlattı (01:17:08 - 01:20:35); yazdırma ve arama çağrıları ise o parçada yoktu.
- Özyineleme yığın (stack) kullanır. Ama ağaç soyut veri tipi (ADT) yığınla değil, bağlı liste ya da dizi ile gerçekleştirilir. Ekleme, silme ve arama özyineleme olmadan da yazılabilir; yalnızca daha uzun olur.

İkili Ağacın Dizide Saklanması

- Kök 0. gözdedir. i. gözdeki düğümün sol çocuğu $2i + 1$, sağ çocuğu $2i + 2$. gözdedir.
- Seyrek bir ağaçta aradaki boş gözler de ayrılmak zorundadır. Örneğin 14. gözde ve onun çocuğu olan 29. gözde eleman varsa aradaki pek çok göz boş kalır.
- Bu yüzden hareketli ve boşluklu yapılarda bağlı liste tercih edilir. Boşluk az olacaksa dizi daha verimlidir.

Bellekteki Heap Bölgesi (Ara Bilgi)

- Bu konu heap ağacıyla ilgili değildir. Yalnızca karışmasın diye anlatıldı.
- Fonksiyon içinde tanımlanan yerel bir dizi `return` ile döndürülürse içindeki değerlere güvenilemez. Fonksiyondan çıkınca o bölge serbest kalır.
- `malloc` ile ayrılan yer heap bölgesindedir ve program çalıştığı sürece (serbest bırakılana kadar) korunur. Bu yüzden fonksiyondan dönen adres geçerli kalır.

Heap Ağacı

- **Dengeli (balanced):** Yaprakların olduğu en alt seviye n ise, 0. seviyeden $n-2$. seviyeye kadar bütün düğümlerin iki çocuğu vardır. $n-1$. seviyede çocuğu olmayan düğüm olabilir.
- **Sola dayalı (left justified):** En alt seviyedeki yapraklar soldan başlayarak boşluksuz dizilir.
- **Heap özelliği:**
  - Max heap örneği (ekrandan, 01:56:00): kök 17; çocukları 9 ve 8; 9'un çocukları 5 ve 7; 8'in çocuğu 3.
  - Bir düğümün bir çocuğu ondan küçük, öbürü büyükse bu ağaç ne max ne de min heap olur.
- Huffman ağacı öncelikli kuyruk (priority queue) yapısıyla kurulur. Öncelikli kuyruk sonraki derslere bırakıldı.

Heap Ağacına Ekleme ve Kökü Silme

Örnek min heap, ekrandan (02:04:30 ve 02:10:00).

```
Ekleme (yeni eleman ilk boş göze konur, sonra yukarı taşınır = sift up)
8 geldi : [8]
4 geldi : [8, 4]      -> 4 < 8, yer değiştir    -> [4, 8]
9 geldi : [4, 8, 9]   -> 9 > 4, dokunma
5 geldi : [4, 8, 9, 5] -> 5 < 8, değiştir -> [4, 5, 9, 8]; 5 > 4, dur
1 geldi : [4, 5, 9, 8, 1] -> 1 < 5, değiştir -> [4, 1, 9, 8, 5]
                          -> 1 < 4, değiştir -> [1, 4, 9, 8, 5]

Kökü silme (kök son elemanla değiştirilir, sonra aşağı taşınır = sift down)
[1, 4, 9, 8, 5] -> 1 ile 5 yer değiştirir, 1 çıkarılır -> [5, 4, 9, 8]
5'in çocukları 4 ve 9; küçüğü 4; 4 < 5, değiştir       -> [4, 5, 9, 8]
5'in çocuğu 8; 5 < 8, dur
```

- Ekleme sadece yeni elemanın yolunu izler. Örneğin 5 eklenirken yalnız 8 ve 4 ile karşılaştırılır, sağ alt ağaca hiç bakılmaz.
- Sift down'da önce iki çocuk birbiriyle karşılaştırılır, sonra seçilen çocuk aşağı inen elemanla karşılaştırılır.
  - 5'in çocukları 8 ve 7 olsaydı: $8 < 7$ değil, küçük olan 7. $5 < 7$ olduğu için 5 yerinde kalır.
- Bir düğümün kaç çocuğu olduğu eleman sayısı n'e bakılarak anlaşılır. $2i + 1 \geq n$ ise düğümün çocuğu yoktur. $2i + 2 \geq n$ ise yalnız sol çocuğu vardır.

Heap Sort

- Max heap ile küçükten büyüğe sıralama:
  1. Diziyi heap haline getir (heapify).
  2. $i = n - 1$'den 1'e kadar: kökü (0. göz) i. gözle değiştir. Sonra ilk i eleman için sift down yap.
- Kod tahtadan (ekrandan, 02:43:30). `siftDown` fonksiyonunun içi bu derste yazılmadı.

```c
void HeapSort(int Heap[], int n)
{
    int i;   /* tahtada tanımı yoktu; derlenmesi için eklendi */
    for (i = n - 1; i > 0; i--)
    {
        swap(&Heap[0], &Heap[i]);
        siftDown(Heap, i);    /* şu anda i elemanım kaldı */
    }
}
```

- i azaldıkça bakılan bölge küçülür. Böylece sona yerleşen elemanlara bir daha bakılmaz.

Örnek 1 (ekrandan, 02:17:50). Başlangıç max heap: `[45, 22, 36, 13, 5, 21, 19, 8]`.

Adım	Değişim	Sift down sonrası heap kısmı	Yerleşmiş kısım
1	45 ile 8	`36, 22, 21, 13, 5, 8, 19`	`45`
2	36 ile 19	`22, 19, 21, 13, 5, 8`	`36, 45`
3	22 ile 8	`21, 19, 8, 13, 5`	`22, 36, 45`
4	21 ile 5	`19, 13, 8, 5`	`21, 22, 36, 45`
5	19 ile 5	`13, 5, 8`	`19, 21, 22, 36, 45`
6	13 ile 8	`8, 5`	`13, 19, 21, 22, 36, 45`
7	8 ile 5	`5`	`8, 13, 19, 21, 22, 36, 45`

- Sonuç: `5, 8, 13, 19, 21, 22, 36, 45`.
- Örneğin 1. adımda 8 köke gelir. Çocukları 22 ve 36'dan büyüğü 36'dır, 8 ile 36 yer değiştirir. Sonra 8'in çocukları 21 ve 19'dan büyüğü 21'dir, 8 ile 21 yer değiştirir.

Örnek 2 (ekrandan, 02:43:30). $n = 5$, max heap: `[22, 19, 15, 8, 11]`.

i	Değişim	Sift down sonrası heap kısmı	Yerleşmiş kısım
4	22 ile 11	`19, 11, 15, 8`	`22`
3	19 ile 8	`15, 11, 8`	`19, 22`
2	15 ile 8	`11, 8`	`15, 19, 22`
1	11 ile 8	`8`	`11, 15, 19, 22`

- Sonuç: `8, 11, 15, 19, 22`.

Karmaşıklık:

- Ağaç dengeli olduğu için yüksekliği $\lfloor \log_2 n \rfloor$'dir.
- Bir elemanı aşağı (sift down) ya da yukarı (sift up) taşımak en kötü durumda $O(\log n)$'dir.
- n elemanı tek tek ekleyerek heap kurmak $O(n \log n)$'dir.
- Sıralama döngüsünde $n - 1$ kez sift down yapılır: $O(n \log n)$.
- Toplam: $O(n \log n) + O(n \log n) = O(n \log n)$. Heap sort, merge sort ile aynı karmaşıklıktadır.

Quiz Sorularının Çözümü

- **Dairesel kuyrukta (circular queue) yeni elemanın yeri:**
  - N elemanlı dizi ile gerçekleştirilen dairesel kuyrukta son elemanın gözü `rear` ise yeni eleman `(rear + 1) % N` gözüne yerleşir.
  - Yalnız `rear + 1` yanlıştır. `rear` dizinin son gözündeyken ve dizinin başı boşalmışken yeni eleman 0. göze gelmelidir.
  - `rear->next` de yanlıştır. Soru dizi ile gerçekleştirilen kuyruğu soruyordu, bağlı listeyi değil.
  - Dairesel yapının amacı: eleman çıkınca herkesi bir öne kaydırmak $O(n)$ sürer. Bunun yerine `front` ilerletilir ve boşalan göz tekrar kullanılır.
- **Bağlı listeyle kuyrukta `isFull`:** Bağlı liste kendi başına dolmaz. Doluluk ancak bellek sınırıyla ilgilidir: eklenen eleman sayısı bir sınırı aşarsa ya da yer ayrılamazsa hata verilir.
- **Yığın ile kuyruk:** Yığında `push`, `pop` ve yığını ters çeviren `reverse` var.
  - Enqueue: `push`.
  - Dequeue: `reverse`, `pop`, `reverse`.
  - Örnek: yığında alttan üste 3, 5, 9, 11 var. `reverse` sonrası 3 üste gelir, `pop` 3'ü alır, ikinci `reverse` yığını 5, 9, 11 sırasına geri getirir.

Ders sonrası notlar

- Hoca, iki çocuklu düğüm silinirken sağ alt ağacın en küçük elemanının "her zaman yaprak" olacağını söyledi. Bu her zaman doğru değildir. Sağ alt ağacın en küçük elemanının sol çocuğu olmaz, ama sağ çocuğu olabilir. Bu durumda o sağ çocuk, taşınan düğümün eski ebeveynine bağlanmalıdır.
- Hoca silme anlatımında bir yerde "sağ alt ağacın kökündeki eleman alınır" dedi, hemen sonra "sağ alt ağacın en küçük elemanı" diye düzeltti. Doğrusu en küçük elemandır.

- Hoca, global değişkenlerin de heap bölgesinde saklandığını ve heap bölgesini CPU'nun düzenlediğini söyledi. Doğrusu: global ve `static` değişkenler veri (data/BSS) bölümündedir. Heap bölgesini işletim sistemi ve `malloc`/`free` gibi bellek ayırma fonksiyonları yönetir.
- Yerel bir dizinin adresini `return` ile döndürmek tanımsız davranıştır (undefined behavior). Derleyici genelde uyarı verir. Bu adres fonksiyon dışında kullanılmamalıdır.
- Slaytta "Heapify the array ⇒ N" yazıyor, anlatımda ise heap kurmanın $O(n \log n)$ olduğu söylendi. İkisi farklı yöntemlere karşılık gelir. Elemanları tek tek ekleyerek (sift up) kurmak $O(n \log n)$'dir. Diziyi aşağıdan yukarıya sift down ile heap yapmak (Floyd yöntemi) $O(n)$'dir. Her iki durumda da heap sort toplamda $O(n \log n)$'dir.
- Derste ekleme örneğinden sonra min heap önce "1, 4, 8, 5, 9" diye okundu (02:04:20), hemen sonra doğru sırayla söylendi (02:04:44). Dizideki gerçek sıra `[1, 4, 9, 8, 5]`'tir (ağaçta 1 kök, 4 ve 9 çocukları, 8 ve 5 de 4'ün çocukları).

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.